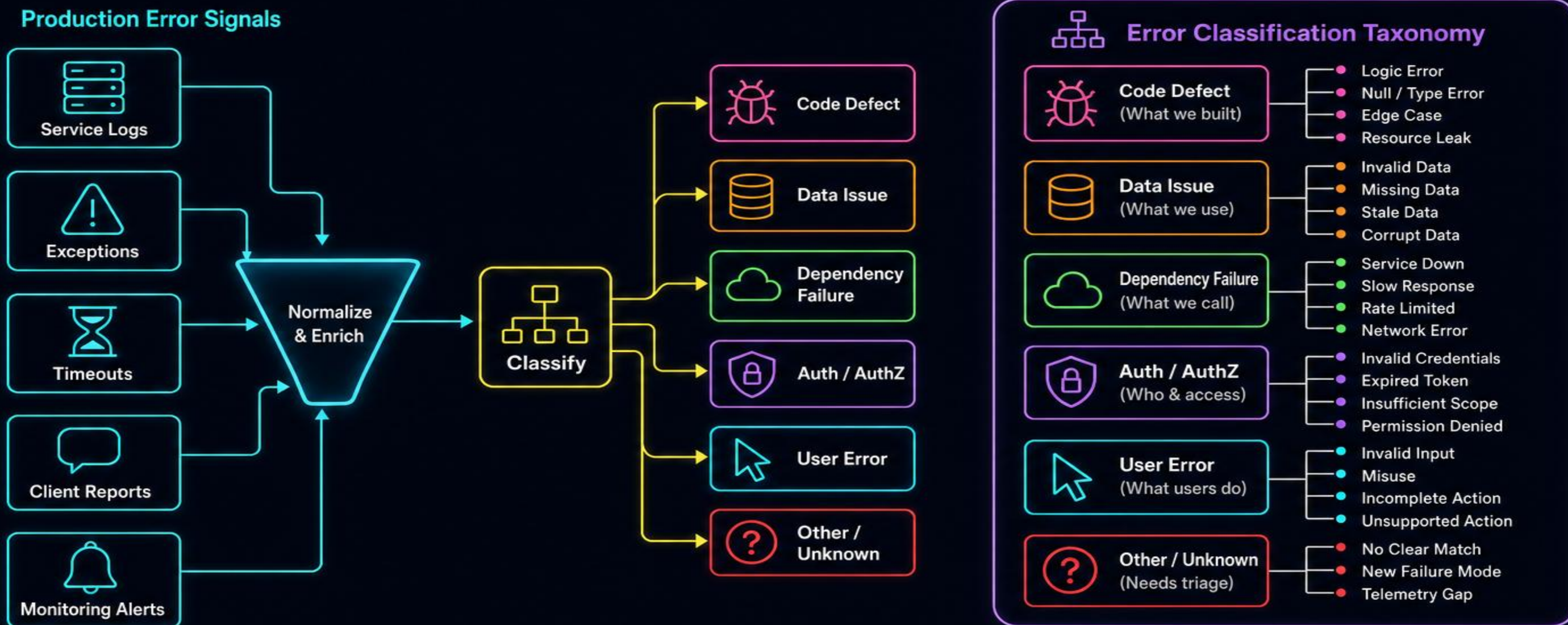
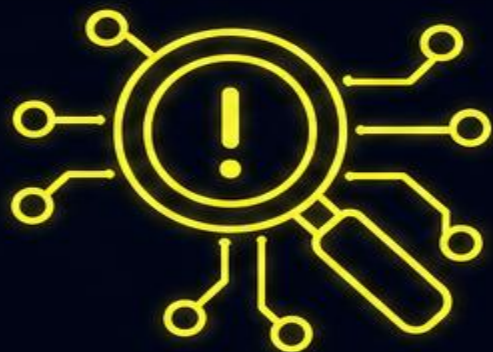


Classifying Errors by Type and Origin

Before you fix anything, you have to name what broke. Classifying the failure is a distinct step that comes before choosing any fix.



Three Terms to Anchor



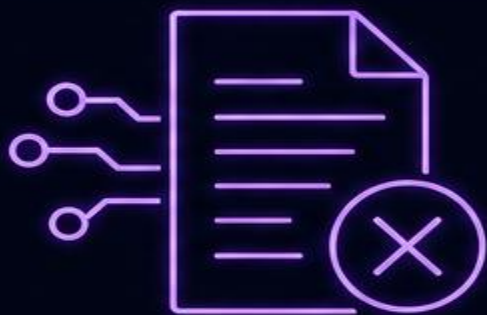
Error Type Identification

The diagnostic step of classifying what kind of failure occurred before choosing a fix. It comes first and decides recovery.



Transient Error

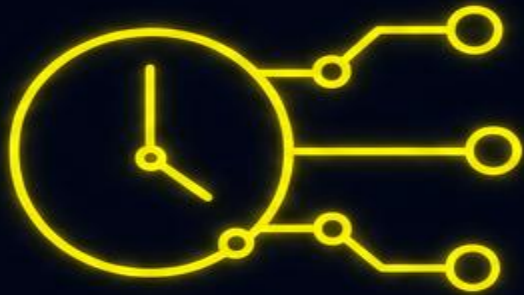
A timeout, rate limit, or 5xx that is expected to clear on its own, so retrying it is reasonable.



Permanent Error

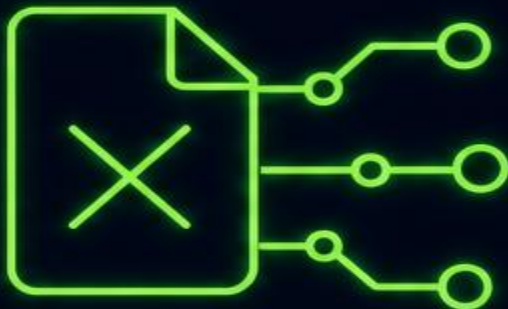
A bad request or bad credentials that will not clear on retry and needs a fix, fallback, or escalation.

Transient or Permanent



Transient

Timeouts, rate limits, network blips, and 5xx server errors. The request was fine and the system was briefly busy, so the same call can succeed later.



Permanent

A 400, 401, 403, 404, or 413. Something about the request itself is wrong, so repeating it unchanged just reproduces the same failure.

How Transient Errors Show Up

Transient errors announce themselves through a handful of familiar signals.



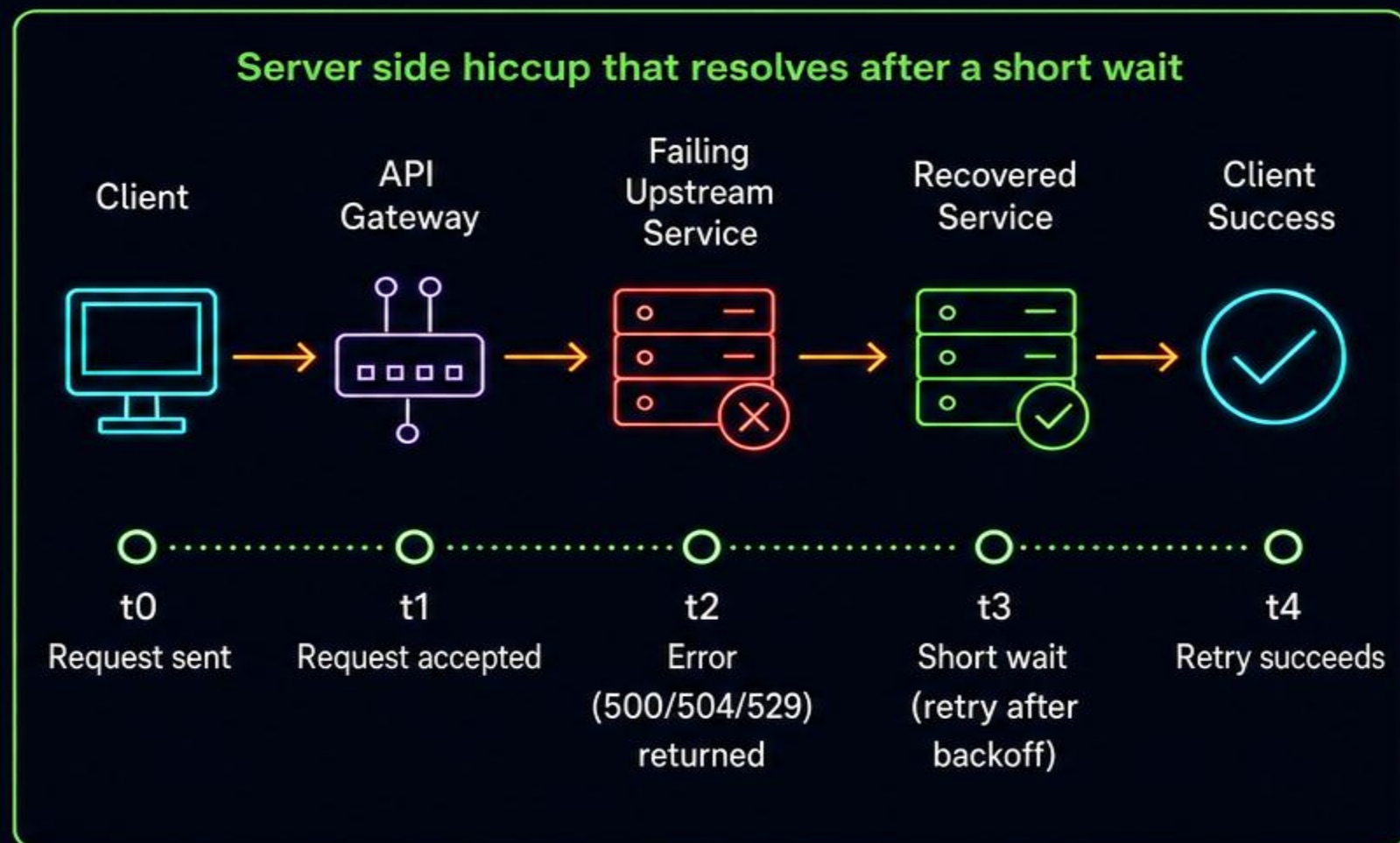
A 429 rate_limit_error when you have sent too many requests



A 500 api_error or a 504 timeout_error from the server side



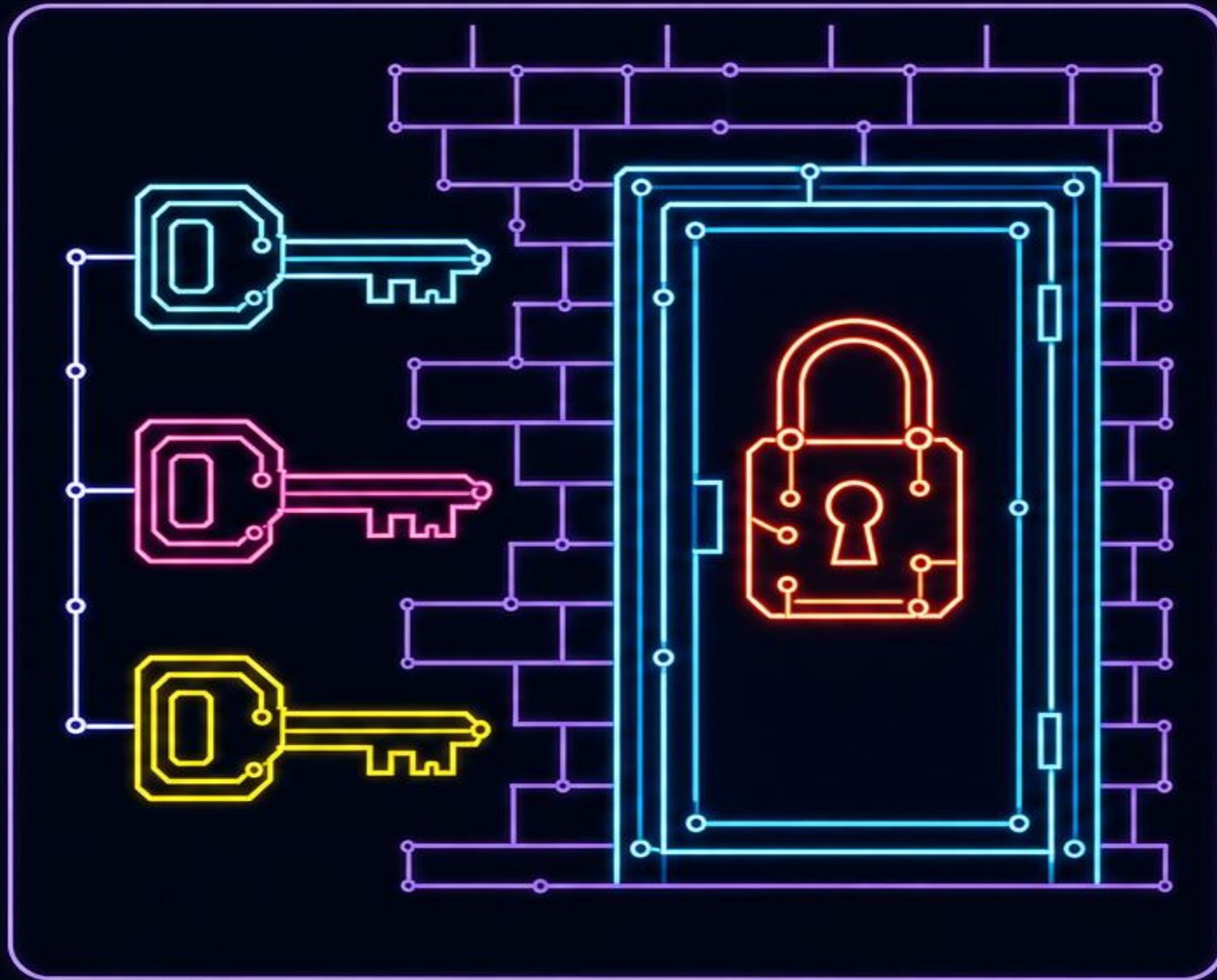
A 529 overloaded_error when the platform is briefly saturated



Each one tends to clear on its own, so a patient retry is the right instinct.

Permanent Errors Won't Retry Away.

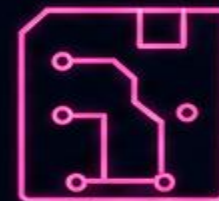
Permanent errors sit in the request itself, so retrying changes nothing.



A 400 `invalid_request_error`
means the payload is malformed



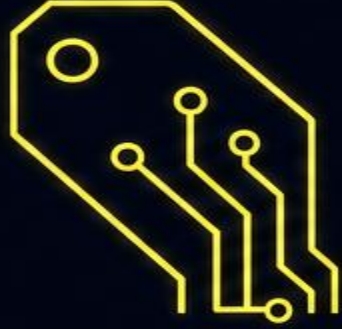
A 401 `authentication_error`
means the key is wrong or expired



A 413 `request_too_large`
means the request exceeds the size limit

Fix the cause, because the same call keeps failing until you do.

Three Parts of an Error Body



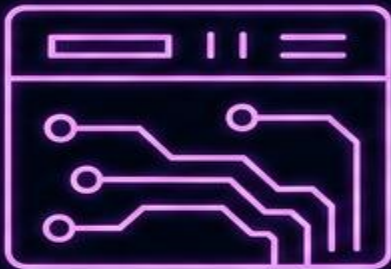
type

A stable machine-readable label like `invalid_request_error` or `overloaded_error` that tells you which class of failure occurred.



message

A human-readable sentence describing the problem, meant for logs and debugging, not for branching your code logic on.



request_id

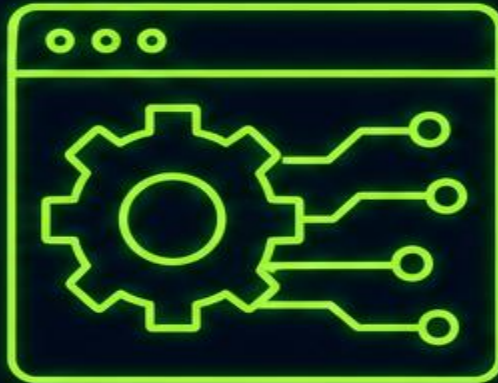
The identifier echoed from the response header. Capture it and include it in any support ticket you open.

Model Output or Integration



Model-output error

The failure lives in what Claude generated, a wrong plan, a bad tool argument, or a truncated answer. This rides inside a successful 200 response.



Integration error

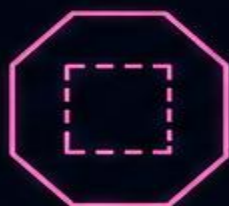
The failure lives in your surrounding code, the request you built, the parsing, the auth, or the endpoint. It usually surfaces as an HTTP status.

The Silent Failure Problem

The trickiest failures make no noise, no exception and no error code.



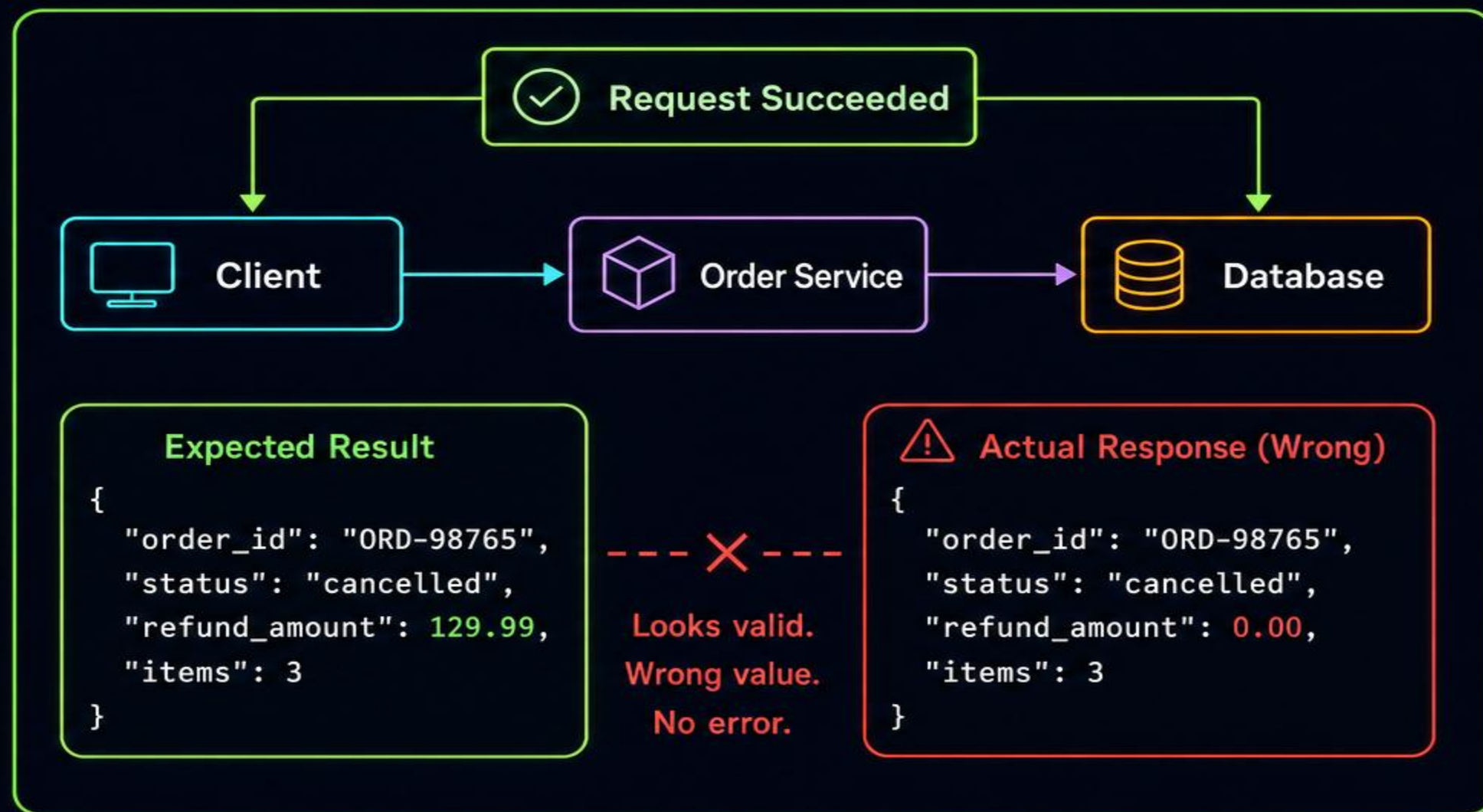
A 200 response whose answer is incomplete or malformed



A stop_reason of max_tokens that quietly truncated the output

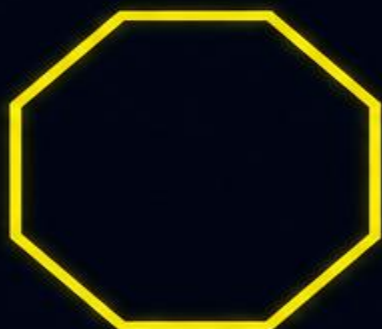


A reply that parses but carries the wrong values



No error fires, so only an output check catches it downstream.

Refusal or Bug



Refusal

Claude intentionally declines on policy or safety grounds. It is valid behavior, not a malfunction, so read **stop_details** and adjust the prompt if the ask is legitimate.



Bug

An unintended failure in your integration code or the model output. It leaves no useful result and needs a real fix, not a reworded prompt.

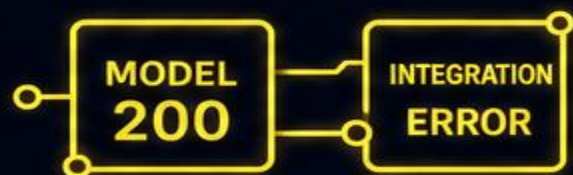
Name It Before You Fix It



Transient errors clear on their own, so retry them



Permanent errors need a fix or a fallback



Model output rides a 200, integration shows a status



Silent failures and refusals both look like a non-answer

Selecting the Right Recovery Strategy

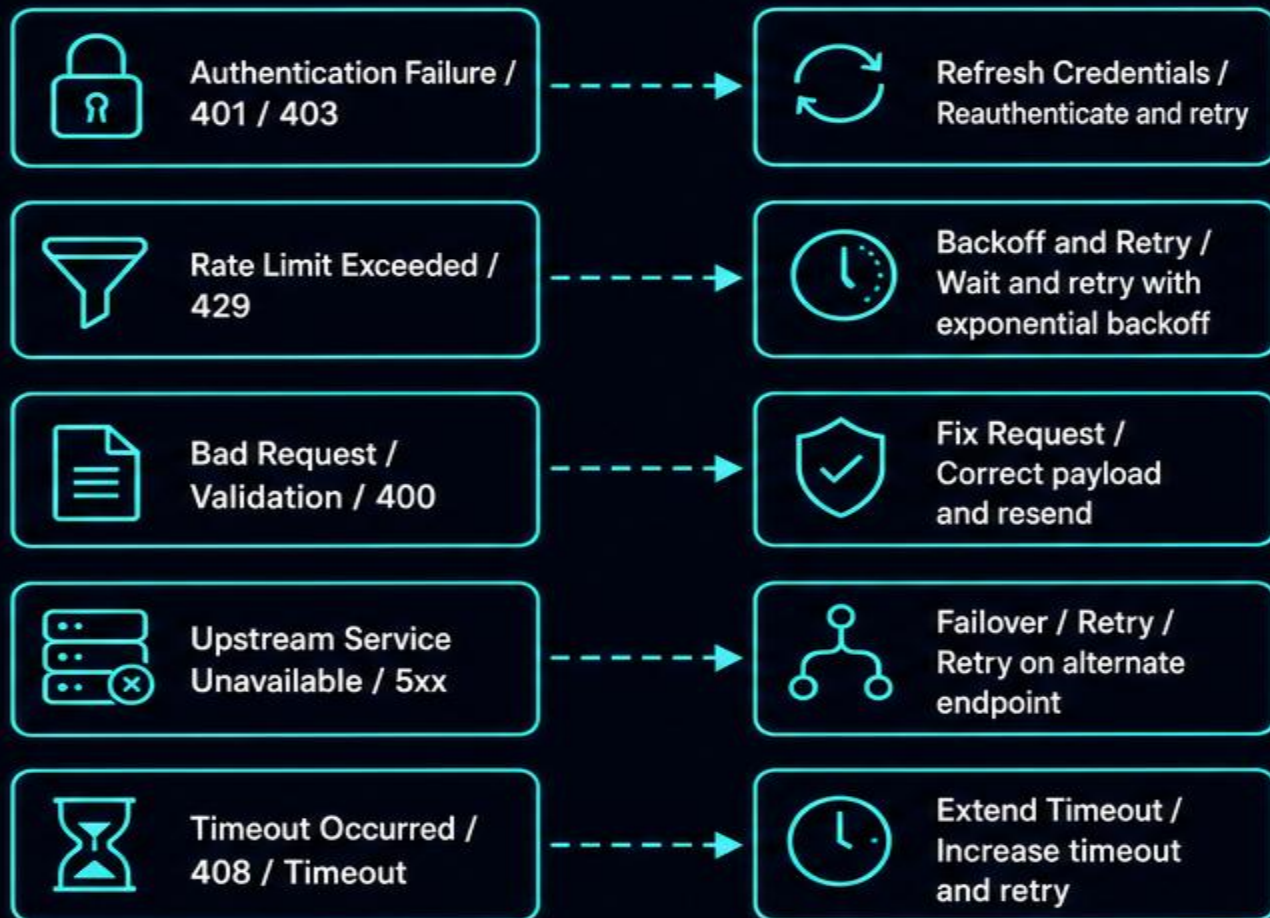
Once you've classified an error, the recovery almost follows from the type.

The trap is matching your fix to the surface symptom instead.

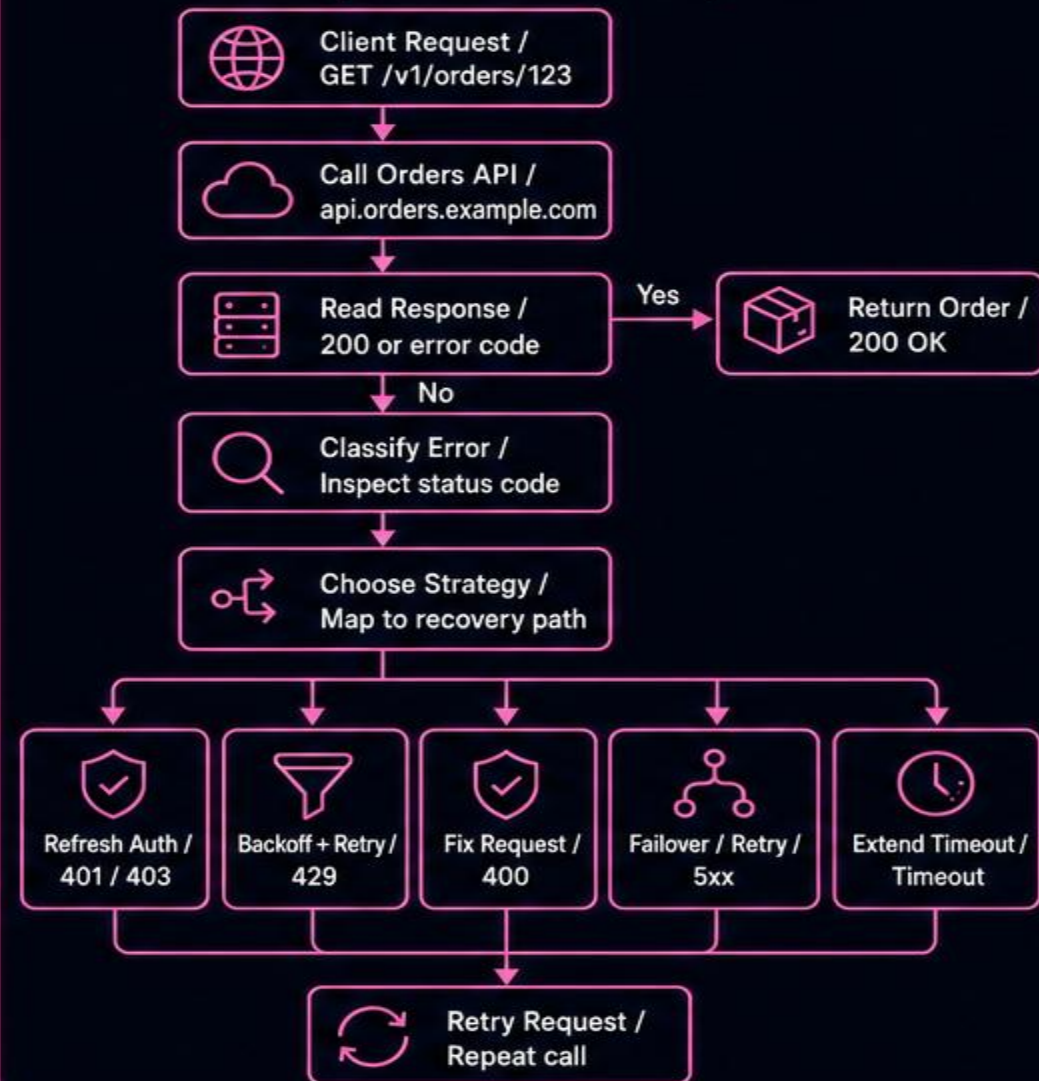
From Error Diagnosis to Recovery Strategy

Error Diagnosis

Recovery Strategy



Example: Order Lookup Workflow



Three Recovery Moves



Retry with Backoff

Re-sending a transient request after an increasing wait. Exponential backoff spaces attempts out so you don't hammer a busy service.



Fallback

Switching to a different model, path, or cached result when the primary call can't succeed, so the user still gets an answer.



Escalation

Handing the failure to a human or a support channel when no automated recovery applies, carrying the `request_id` for context.

Match the Type, Not the Symptom

The recovery follows the error's type, never its surface appearance.



A slow response and a rate limit both feel like waiting

They look the same from the user's point of view, but the causes are completely different.



One needs patience, the other needs a real backoff schedule

The right response depends on the error's nature, not how it feels.



Retrying a bad request feels productive and fixes nothing

Some errors are permanent until the request is changed.

SYMPTOM

Still waiting...



Actual Error
Slow Response



SYMPTOM

Still waiting...



Actual Error
Rate Limit Exceeded



RESPONSE



Be Patient
Allow more time

RESPONSE



Backoff Schedule
Retry later with delay



TIP: Different errors need different recovery strategies.
Handle the cause, not the feeling.

Three Recovery Buckets



Retry and back off

For transient integration errors, connection drops, 429, and 5xx. Let the wait grow and honor retry-after.



Fix the request

For permanent errors like 400, 401, or 413. Change the payload, credentials, or size, then send the corrected call.



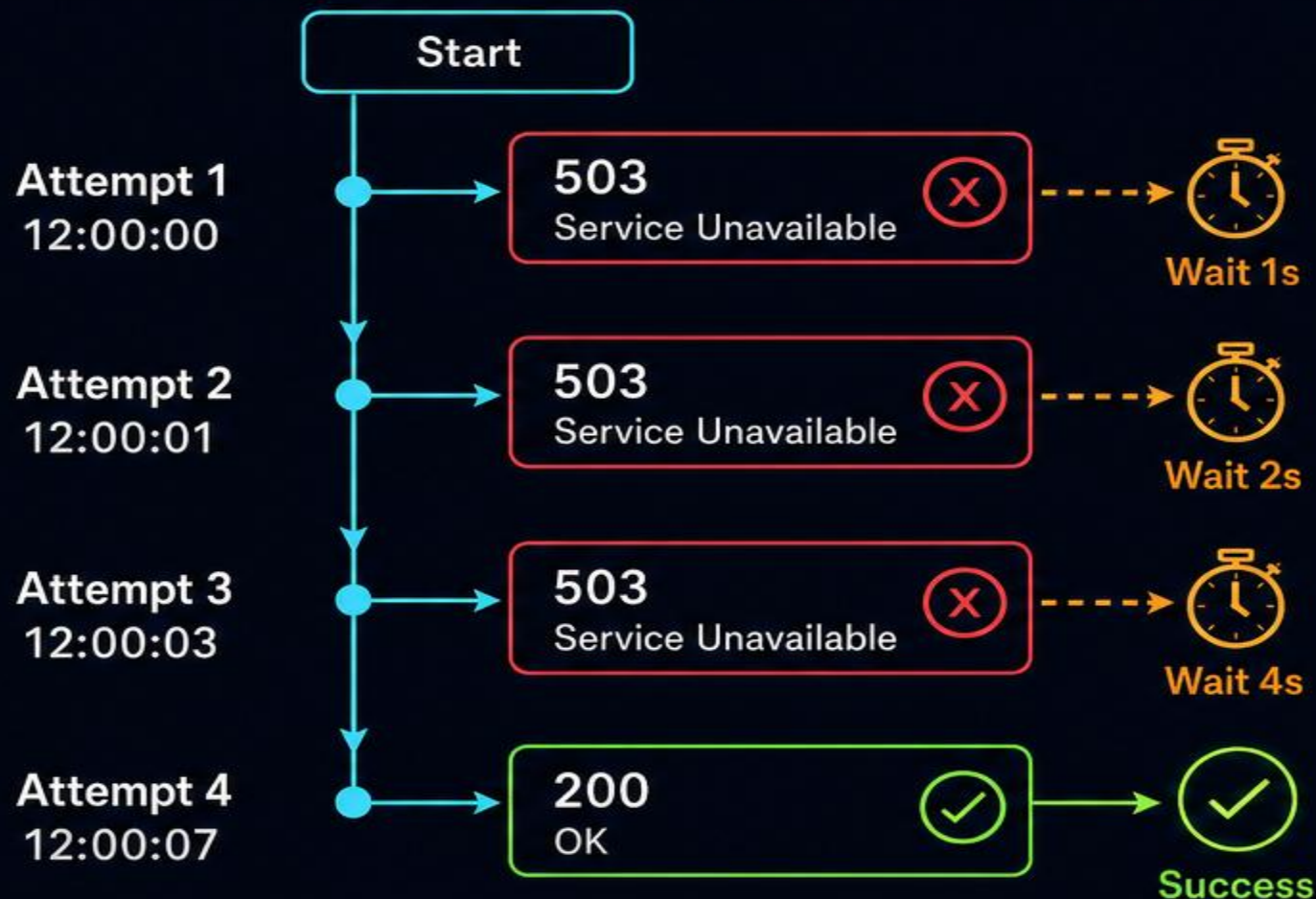
Fix the prompt or fall back

For model-output problems on a 200. Adjust the prompt, raise a limit, or switch models.

Backoff Belongs to Transient Errors

Exponential backoff is the right tool for one job, transient failures.

API Request with Exponential Backoff



Total elapsed time: ~7 seconds
Delays: 1s + 2s + 4s = 7s of waiting



Wait a little, then wait longer on each successive attempt



On a 429, honor the retry-after header instead of guessing



Cap the attempts so you fail loudly, not forever

What the SDK Retries



Retried for you

The official SDKs automatically retry transient failures, connection errors, 429s, and 5xx, with exponential backoff, honoring retry-after. It's a couple of retries by default and configurable.



On you to handle

The permanent 4xx client errors, 400, 401, 403, and 413, are never auto-retried, with 429 the transient exception. A 409 conflict is resolve-first, then retry.

Fixing Permanent Errors

Permanent errors clear only when you change the request or its context.



A 400,
read the message, then
fix the malformed field



A 401 or 403,
repair the key or its
permissions



A 413,
shrink the request or
split it into batches

A 409 is different, resolve the
conflict first, then retry.



Fix



Validate



Send



Success

Bad Request

```
{
  "model": "claude-opus-5",
  "max_tokens": "4096",
  "messages": [
    {
      "role": "user",
      "content": "Summarize\nQ3 results."
    }
  ]
}
```



400 Bad Request

invalid type for max_tokens:
expected integer

Corrected Request

```
{
  "model": "claude-opus-5",
  "max_tokens": 4096,
  "messages": [
    {
      "role": "user",
      "content": "Summarize\nQ3 results."
    }
  ]
}
```



200 OK

Request accepted

Four Model-Output Fixes



max_tokens

The answer was truncated at your cap. Raise max_tokens or continue the response where it stopped.



model_context_window_exceeded

The response hit the model's total context window, not your requested cap. Reduce input, prune history, or compact.



refusal

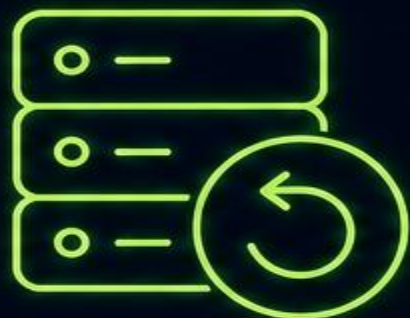
Claude declined on policy grounds. Read stop_details, adjust the prompt, or fall back to another model.



wrong but valid

A clean answer that's simply incorrect. A prompt and eval problem, not a network retry.

Fallback or Escalate



Fallback

An automated plan B. Switch to another model, a simpler path, or a cached result so the user still gets something useful.



Escalate

No automatic recovery fits, so route the failure to a human or a support channel, carrying the request_id and the error type for context.

Let the Type Choose the Fix



Retry with backoff for transient errors only



Fix the request for permanent 4xx errors



Adjust the prompt or fall back for model output



Escalate with the request_id when nothing automated fits

Trace Analysis and Origin Isolation

Every failure surfaces somewhere in the trace, the ordered record of prompts, tool calls, results, and errors.



Three Debugging Foundations



Trace Analysis

Reading the ordered execution record, prompts, tool calls, tool results, thinking, and errors, to reconstruct what happened and where behavior diverged from intent.



Problem-Origin Isolation

Localizing a failure to one layer, your integration code or the model output, before editing anything, so the fix lands on the right component.



Origin Evidence

The mid-trace tell that points to a layer, a wrong tool argument, an empty tool result, or a truncated response.

Integration Layer vs Model Output



Integration Layer

Everything wrapping the model, the API request, endpoint, auth, response parsing, and your tool harness. Failures here show up as HTTP 4xx or 5xx codes, or as transport errors in the event stream.



Model Output

The generation itself, riding on a successful 200 response. Problems appear in the content or the stop_reason, a wrong plan, a misread result, or a truncated answer.

Reading the Trace Outward-In

Work outward-in, one checkpoint at a time.



Check the HTTP status first, a 4xx or 5xx means integration



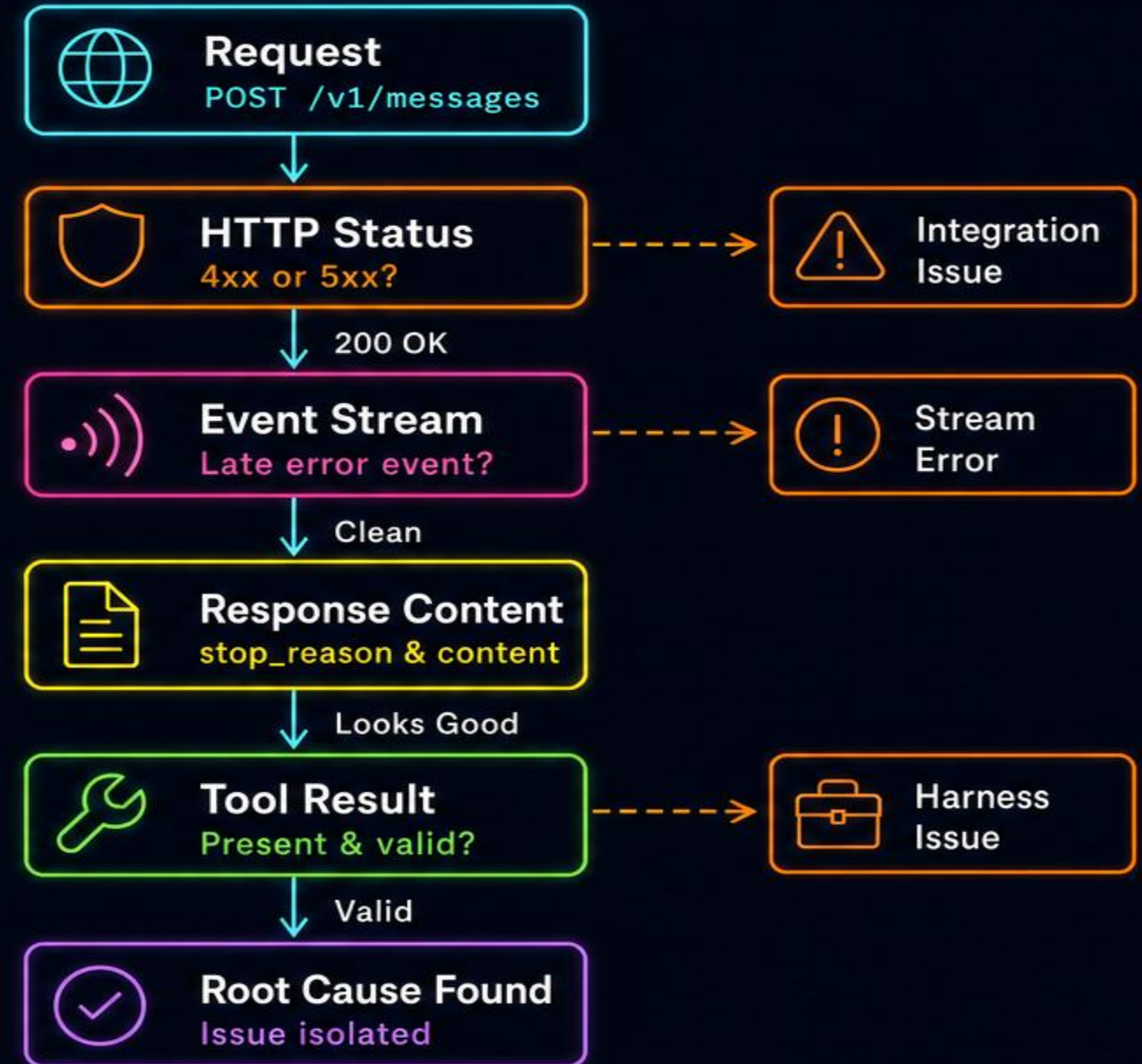
On a 200, read the stream for a late error event



On a clean 200, inspect stop_reason and the content



A thrown or empty tool result points back at your harness



Where Failures Announce Themselves



HTTP Status Code

A 4xx or 5xx status means the request never produced a usable answer. Branch on the error type.



SSE Error Event

After a 200, a streamed error event still means integration failure. Read the event stream to catch it.



stop_reason

On a clean 200, this field is the model-output tell, truncated, refused, or finished with wrong content.

Three Model-Output Signals



max_tokens

The response hit the max_tokens cap you set, so the answer is truncated but valid. Raise the cap or continue.



model_context_window_exceeded

The response filled the model's total context window, not your requested cap. Prune history or compact context.

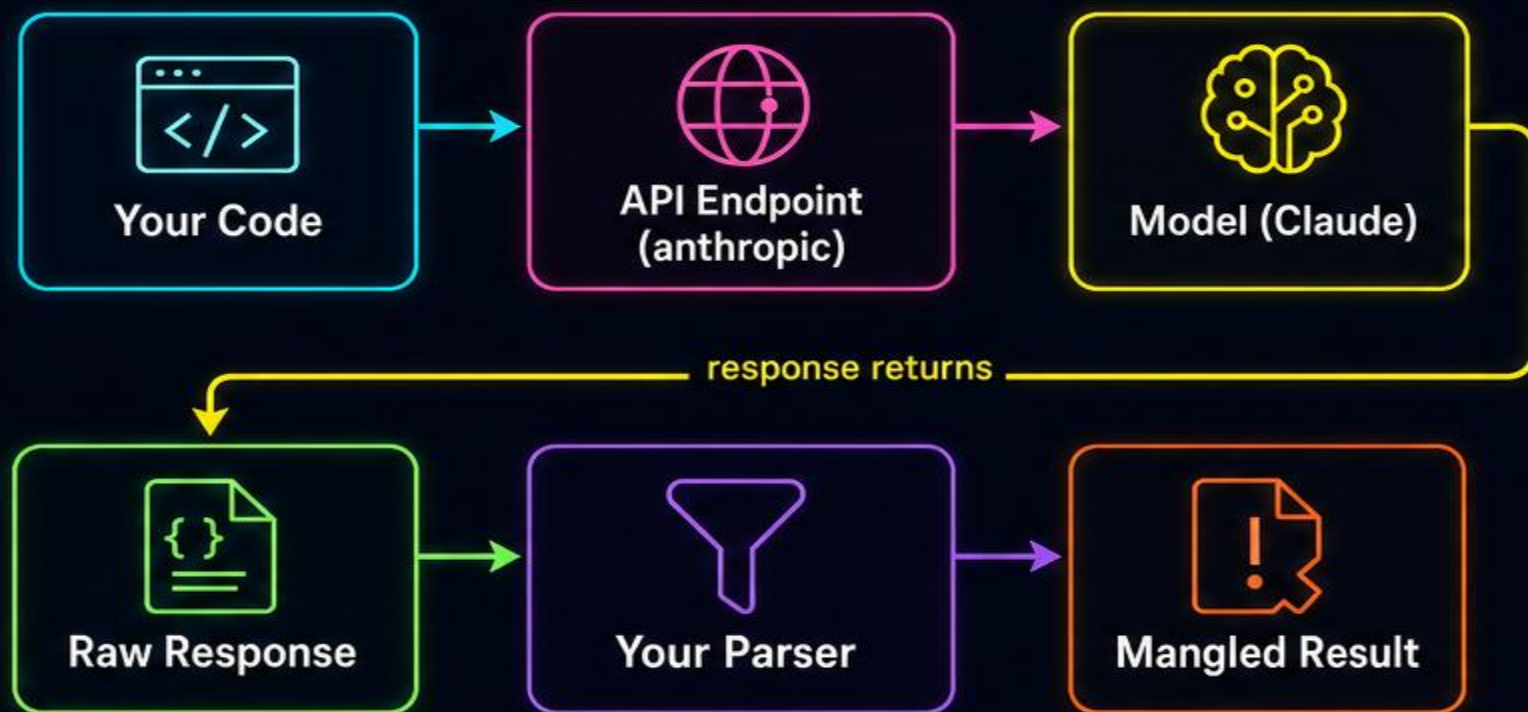


refusal

Claude declined on policy grounds, with stop_details carrying the category. Adjust the prompt or fall back.

Integration-Layer Culprits

These live in your code, not the model, even when the answer looks wrong.



A malformed request or the wrong endpoint

Bad paths, headers, params, or body shape can cause rejections or model fallbacks.



A response you parsed or mangled incorrectly

Regex slips, JSON assumptions, or encoding mismatches lead to corrupted or partial data.



Auth, billing, or rate-limit failures in the wrapper

Expired tokens, quota limits, or payment issues surface as errors or degraded behavior.



A tool that threw or returned an empty result

Exceptions, timeouts, or silent failures leave the agent with nothing useful to proceed on.

CONFIGURE GENERATION PARAMETERS

```
response = client.messages.create(  
    model="claude-opus-5",  
    max_tokens=1024, # cap length to control cost and truncation  
    stop_sequences=["\n\nHuman:", "\n\nUser:"], # stop before role prefixes in transcripts  
    messages=messages,  
)
```

Model-Output Culprits

These originate in the generation, even though the integration worked fine.



A wrong plan despite sufficient information

The model picks an unnecessary detour or the wrong sequence.



A misread tool result feeding a bad next step

The model misinterprets a correct result and acts on a false premise.



A bad tool argument the model chose

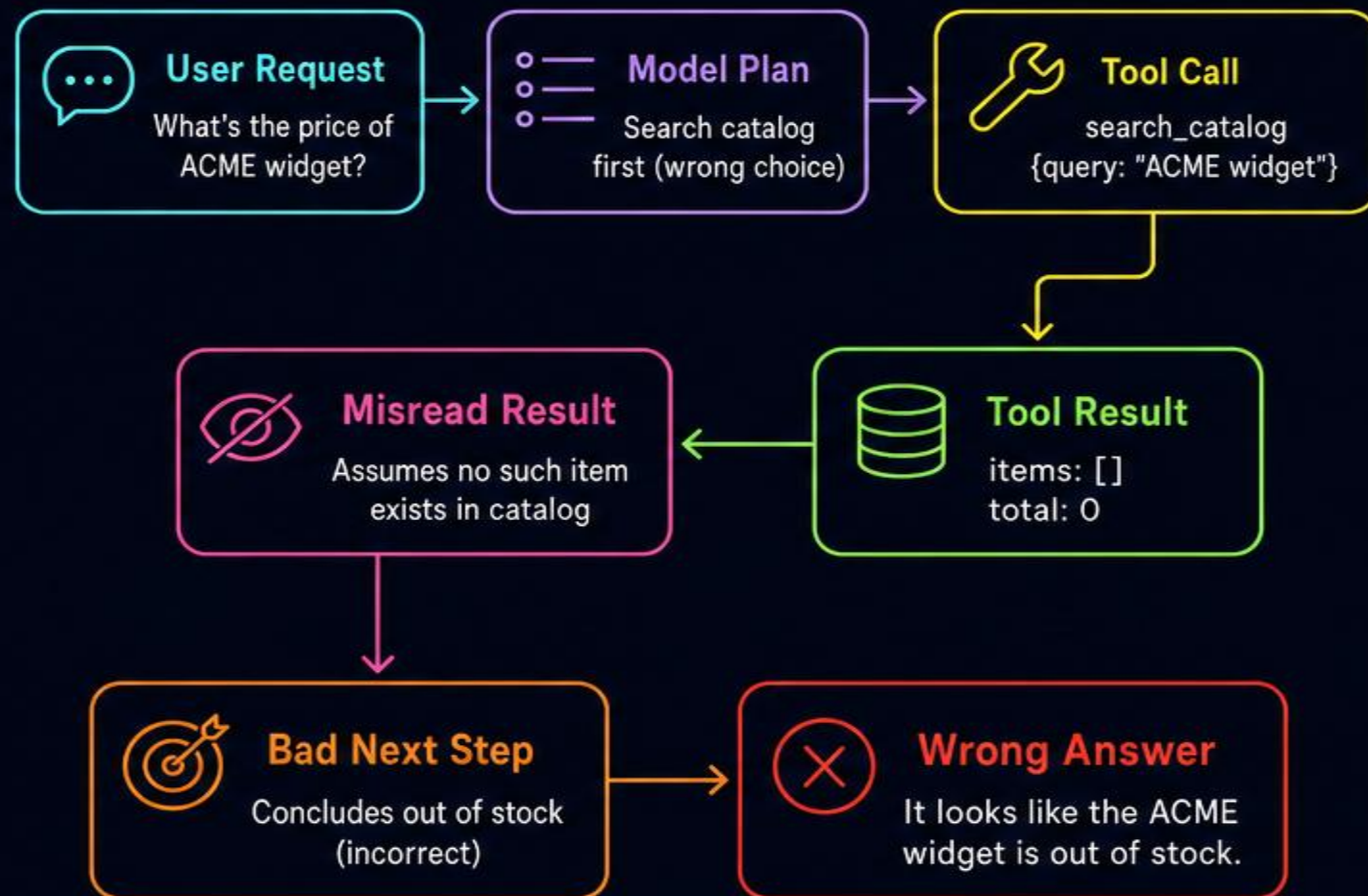
The model supplies incorrect, missing, or risky parameters to a tool.



A refusal or a truncated, incomplete answer

The model declines when it should answer or cuts off before finishing the task.

EXAMPLE: PRICE CHECK GONE WRONG



Fix these with prompt work and evals, not a retry.

Two Ways a Trial Fails



Agent Made the Mistake

The transcript shows genuinely flawed reasoning, a wrong tool call, or a misread result. The generation owns the failure, so fix the prompt or model.



Grader Rejected a Valid Answer

The agent was actually right and your grader or harness marked it wrong. That is an integration bug in your eval, not a model failure.

Isolate the Layer, Then Fix



The trace is your primary evidence, not the final output



HTTP codes and stream errors mean the integration layer



stop_reason on a 200 means the model-output layer



Transcripts reveal whether the agent or the grader failed